

Лабораторная работа №2

Когда индексов недостаточно: производительность PostgreSQL при росте данных

1. Цель работы

В предыдущей лабораторной работе вы изучили индексы и научились использовать `EXPLAIN ANALYZE` для анализа производительности SQL-запросов.

В этой работе мы сделаем следующий шаг:

Что произойдёт с базой данных, если количество данных увеличится в десятки и сотни раз?

В рамках работы необходимо исследовать, как изменение объёма данных влияет на:

- время выполнения запросов;
- планы выполнения запросов;
- размер таблиц;
- размер индексов;
- операции `INSERT`, `UPDATE` и `SELECT`;
- эффективность индексов на больших объёмах данных.

В конце работы необходимо применить полученные знания к **собственному сервису**.

2. Что необходимо сделать

В лабораторной работе вы будете работать в два этапа:

Часть А — экспериментальная база

Создать отдельную таблицу, сгенерировать большое количество данных и провести серию измерений.

Часть В — собственный сервис

Выбрать одну из сущностей своего сервиса, которая потенциально может содержать большое количество записей, и провести аналогичное исследование.

3. Подготовка

Для выполнения работы вам потребуется:

- PostgreSQL;
- ваш SQL-клиент;
- проект из предыдущих лабораторных работ;
- Docker / Docker Compose;
- доступ к своей базе данных.

Если PostgreSQL запускается в Docker, все эксперименты выполняются внутри вашей PostgreSQL-базы в отдельной схеме.

Часть А. Исследование роста данных

Задание 1. Создание экспериментальной таблицы

Создайте таблицу events:

```
CREATE TABLE events (  
    id BIGSERIAL PRIMARY KEY,  
    user_id BIGINT NOT NULL,  
    event_type VARCHAR(50) NOT NULL,  
    payload JSONB,  
    created_at TIMESTAMP NOT NULL  
);
```

Таблица моделирует поток событий, который постоянно увеличивается.

Задание 2. Генерация данных

Сгенерируйте несколько объёмов данных.

Минимальные контрольные точки:

- 10 000 записей;
- 100 000 записей;
- 1 000 000 записей;
- 5 000 000 записей.

Если позволяет свободное место, то дополнительно:

- 10 000 000 записей.

Для генерации можно использовать `generate_series()`.

Пример:

```
INSERT INTO events (  
    user_id,  
    event_type,  
    payload,  
    created_at  
)  
SELECT  
    (random() * 100000)::bigint,  
    CASE  
        WHEN random() < 0.4 THEN 'MESSAGE'  
        WHEN random() < 0.7 THEN 'LOGIN'  
        WHEN random() < 0.9 THEN 'PURCHASE'  
        ELSE 'OTHER'  
    END,  
    '{} '::jsonb,  
    NOW() - (random() * INTERVAL '365 days')  
FROM generate_series(1, 100000);
```

Количество генерируемых записей изменяйте самостоятельно.

Задание 3. Измерение размера таблицы

Для каждого объёма данных определите размер таблицы.

Используйте:

```
SELECT pg_size_pretty(  
    pg_relation_size('events')  
);
```

Также определите общий размер таблицы вместе с индексами:

```
SELECT pg_size_pretty(  
    pg_total_relation_size('events')  
);
```

Заполните таблицу

Количество строк	Размер таблицы	Общий размер
10 000		
100 000		
1 000 000		
5 000 000		

Количество строк	Размер таблицы	Общий размер
10 000 000		

Если какой-либо объём данных вы не смогли создать, укажите это в отчёте.

Задание 4. Исследование SELECT без дополнительного индекса

Выполните запрос:

```
SELECT *
FROM events
WHERE user_id = 123;
```

Исследуйте его с помощью:

```
EXPLAIN ANALYZE
SELECT *
FROM events
WHERE user_id = 123;
```

Зафиксируйте:

- тип Scan;
- actual rows;
- Rows Removed by Filter;
- Execution Time.

Повторите эксперимент для разных объёмов данных

Количество строк	Scan	Actual Rows	Rows Removed	Execution Time
10 000				
100 000				
1 000 000				
5 000 000				

Вопрос

Как изменилось время выполнения запроса при увеличении объёма таблицы?

Сделайте краткий вывод.

Задание 5. Добавление индекса

Создайте индекс:

```
CREATE INDEX idx_events_user_id  
ON events(user_id);
```

Снова выполните:

```
EXPLAIN ANALYZE  
SELECT *  
FROM events  
WHERE user_id = 123;
```

Сравните новый план со старым.

Обратите внимание на:

- тип Scan;
 - Execution Time;
 - количество обработанных строк;
 - наличие Index Cond.
-

Заполните таблицу

Объём	Без индекса	С индексом	Разница
100 000			
1 000 000			
5 000 000			

Вопросы

1. Как изменился план запроса?
 2. Как изменилось время выполнения?
 3. На каком объёме данных разница стала заметной?
 4. Всегда ли индекс является оптимальным решением?
-

Задание 6. Поиск по диапазону дат

Выполните запрос:

```
SELECT *  
FROM events  
WHERE created_at >= NOW() - INTERVAL '1 day';
```

Исследуйте:

```
EXPLAIN ANALYZE  
SELECT *  
FROM events  
WHERE created_at >= NOW() - INTERVAL '1 day';
```

Создайте индекс:

```
CREATE INDEX idx_events_created_at  
ON events(created_at);
```

Повторите эксперимент.

Вопрос

Почему индекс может давать хороший результат для небольшого диапазона, но быть менее полезным, если запрос возвращает большую часть таблицы?

Задание 7. Фильтрация и сортировка

Исследуйте запрос:

```
SELECT *  
FROM events  
WHERE user_id = 123  
ORDER BY created_at DESC  
LIMIT 100;
```

Выполните:

```
EXPLAIN ANALYZE  
SELECT *  
FROM events  
WHERE user_id = 123  
ORDER BY created_at DESC  
LIMIT 100;
```

Проанализируйте план.

Обратите внимание, присутствует ли операция:

Sort

Попробуйте улучшить запрос

Создайте составной индекс:

```
CREATE INDEX idx_events_user_created
ON events(user_id, created_at DESC);
```

Снова выполните EXPLAIN ANALYZE.

Ответьте:

1. Изменился ли план?
 2. Исчезла ли операция Sort?
 3. Почему порядок колонок в индексе имеет значение?
-

Задание 8. Агрегация

Исследуйте запрос:

```
SELECT
    event_type,
    COUNT(*)
FROM events
WHERE created_at >= NOW() - INTERVAL '30 days'
GROUP BY event_type;
```

Выполните:

```
EXPLAIN ANALYZE
SELECT
    event_type,
    COUNT(*)
FROM events
WHERE created_at >= NOW() - INTERVAL '30 days'
GROUP BY event_type;
```

Ответьте:

1. Какие операции выполняются в плане?
 2. Какой объём данных необходимо обработать?
 3. Помогает ли индекс на created_at?
 4. Может ли индекс полностью решить проблему производительности данного запроса?
-

Задание 9. Стоимость индексов

Теперь исследуем не только SELECT, но и операции записи.

Сравните время выполнения массовой вставки:

```
INSERT INTO events (...)  
SELECT ...  
FROM generate_series(...);
```

до и после создания нескольких индексов.

У вас должны присутствовать как минимум:

```
idx_events_user_id  
idx_events_created_at  
idx_events_user_created
```

Вопросы

1. Как изменилось время INSERT?
2. Почему наличие индексов может замедлять операции записи?
3. Что происходит с индексами при добавлении новых строк?

Сделайте вывод:

Почему нельзя просто создать индекс на каждой колонке таблицы?

Задание 10. Исследование размера индексов

Определите размер созданных индексов.

Например:

```
SELECT  
    indexrelname,  
    pg_size_pretty(pg_relation_size(indexrelid)) AS index_size  
FROM pg_stat_user_indexes  
WHERE relname = 'events';
```

Запишите результаты.

Индекс	Размер
idx_events_user_id	
idx_events_created_at	
idx_events_user_created	

Вопрос

Как изменяется размер индекса при росте количества записей?

Задание 11. Когда индекс уже не спасает?

Рассмотрите запрос:

```
SELECT
    DATE(created_at),
    COUNT(*)
FROM events
WHERE created_at >= NOW() - INTERVAL '365 days'
GROUP BY DATE(created_at);
```

Предположим, таблица содержит 100 миллионов записей.

На created_at существует индекс.

Ответьте письменно:

1. Почему запрос всё равно может быть дорогим?
2. Какой объём данных необходимо обработать?
3. Почему наличие индекса не гарантирует мгновенное выполнение запроса?
4. Какие решения можно рассмотреть, если таблица продолжит расти?

Можно привести несколько вариантов.

Часть В. Исследование собственного сервиса

Теперь необходимо провести аналогичное исследование на **своём проекте**.

Задание 12. Выбор сущности

Выберите одну сущность своего сервиса, которая потенциально может содержать большое количество записей.

Примеры:

messages
orders
events
transactions
comments

notifications
logs
reviews
payments

Если у вас другая предметная область — выберите подходящую сущность самостоятельно.

В отчёте укажите:

- название сущности;
 - количество записей сейчас;
 - почему именно эта сущность была выбрана;
 - почему она потенциально может быстро расти.
-

Задание 13. Выбор запросов

Выберите минимум **три реальных запроса** вашего сервиса.

Среди них должны быть разные сценарии.

Например:

Запрос 1

Поиск по идентификатору или внешнему ключу.

```
SELECT ...  
FROM messages  
WHERE user_id = $1;
```

Запрос 2

Поиск по диапазону дат.

```
SELECT ...  
FROM messages  
WHERE created_at >= $1  
      AND created_at < $2;
```

Запрос 3

Фильтрация + сортировка:

```
SELECT ...  
FROM messages  
WHERE user_id = $1  
ORDER BY created_at DESC  
LIMIT 50;
```

Можно использовать реальные SQL-запросы из вашего Repository / Dapper / EF Core.

Задание 14. Baseline

Для каждого выбранного запроса выполните:

EXPLAIN ANALYZE

Зафиксируйте:

- исходный SQL;
 - план выполнения;
 - Execution Time;
 - используемые индексы;
 - основные операции плана.
-

Задание 15. Увеличение объёма данных

Увеличьте количество данных выбранной сущности.

Минимально необходимо провести эксперимент хотя бы с тремя объёмами:

100k

1m

5m

или другие значения, подходящие для вашей системы.

Задание 16. Сравнение производительности

Для выбранных запросов сравните производительность.

Запрос	Объём данных	Execution Time	Scan	Индекс
№1				
№1				
№2				
№2				
№3				
№3				

Задание 17. Найдите узкое место

На основании полученных результатов определите:

Какой запрос сильнее всего зависит от роста количества данных?

Для него приведите:

1. SQL-запрос.
 2. План EXPLAIN ANALYZE.
 3. Проблемное место.
 4. Объяснение, почему запрос становится дорогим.
-

Задание 18. Попробуйте улучшить запрос

Попробуйте оптимизировать найденный запрос.

В зависимости от ситуации можно:

- изменить индекс;
- создать составной индекс;
- изменить порядок колонок в индексе;
- изменить SQL-запрос;
- уменьшить количество обрабатываемых данных;
- использовать другой способ получения данных.

После изменения снова выполните:

EXPLAIN ANALYZE

Сравните результаты.

Задание 19. Проведите границу применимости индексов

Ответьте на главный вопрос лабораторной:

В какой момент индексов становится недостаточно?

Необходимо привести ответ, основанный на ваших измерениях.

Нельзя отвечать только теоретически.

Пример формата:

При объёме около X млн записей запрос Y начинает выполнять операцию Z и занимает около N секунд. Добавление индекса уменьшило время до M секунд,

однако дальнейший рост данных снова увеличивает время выполнения. Поэтому для данного сценария одного индекса недостаточно.

Контрольные вопросы

Перед сдачей работы вы должны уметь ответить на вопросы:

1. Почему Seq Scan не всегда является плохим планом?
2. Почему PostgreSQL может не использовать существующий индекс?
3. Что происходит с индексом при INSERT?
4. Почему большое количество индексов может замедлять запись?
5. Что такое селективность?
6. Почему индекс хорошо работает для поиска небольшой части таблицы?
7. Почему индекс может быть менее полезен, если запрос возвращает большую часть таблицы?
8. Зачем нужен EXPLAIN ANALYZE?
9. Что означает Rows Removed by Filter?
10. Что означает Index Cond?
11. Чем Index Scan отличается от Bitmap Index Scan?
12. Почему порядок колонок в составном индексе имеет значение?
13. Почему запрос с GROUP BY может оставаться дорогим даже при наличии индекса?
14. Как статистика влияет на выбор плана?
15. Что происходит с производительностью при росте таблицы?
16. Почему нельзя создавать индексы на все колонки?
17. Как определить размер таблицы в PostgreSQL?
18. Как определить размер индекса?